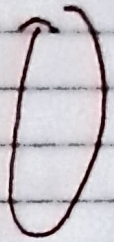
Design and Analysis of Software Systems

Development Cycle:- Dev Life Cycle [SDLC]
Software

↳ Determine whether the proposed sys is viable

(i) Feasibility Analysis:- Req for proposal
(ii) Requirements / Specifications:- What is needed, ~~How to~~
(iii) Design:- How to do
(iv) Implementation:- Modelling
(v) Testing:- Static and Dynamic & Deployment
(vi) Main Maintenance:-



Greenfield & Brownfield systems:-
↓ ↓
Develop from scratch | Work on existing systems

Maintenance is max effort

Modules are never integrated in one shot, multiple steps. During each step, the partially integrated system is tested

Stuff keeps changing, products must evolve

Process Model:- Types of SDLC, SDLC models
(i) Traditional → Plan driven
(ii) Agile → Dynamic

Processes must continually improve

Debriefing - Planning & Execution

Convergence:- Estimates get more accurate as the rounds

Design and Analysis of Software Systems

Development Cycle :- Dev life Cycle [SDLC]

↳ Determine whether the proposed sys is viable

(i) Feasibility Analysis :- Req for proposal

(ii) Requirements / Specifications :- What is needed, ~~How to do~~

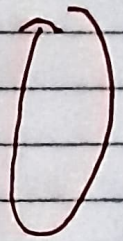
(iii) Design :- How to do

Modelling

(iv) Implementation :-

(v) Testing :- Static and Dynamic & Deployment

(vi) ~~Main~~ Maintenance :-



Greenfield & Brownfield systems :-

↓ ↓

Develop from scratch — Work on existing systems

Maintenance is max effort

Modules are never integrated in one shot, multiple steps. During each step, the partially integrated system is tested

Stuff keeps changing, products must evolve

Process Model :- Types of SDLC, SDLC models

(i) Traditional → Plan driven

(ii) Agile → Dynamic

Processes must continually improve

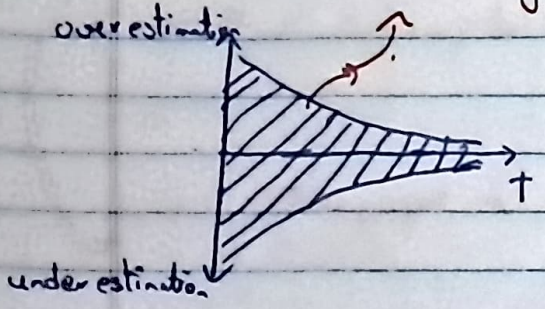
Debriefing - Planning & Execution

Convergence :- Estimates get more accurate as the rounds

progressed

Upfront Planning:-

Cone of Uncertainity



Traditional:- No iterations, one planning session and no changes

Agile:- Many iterations, multiple shorter planning sessions between iterations

Team Dynamics & Self-organization

Leadership and Psychological Safety

Lean Principles

Work in Progress and The Definition of Done
when is the unit
actually done

Rework:- Starting from scratch once again
results
Technical Debt:-

PDCA – Continuous improvement mindset

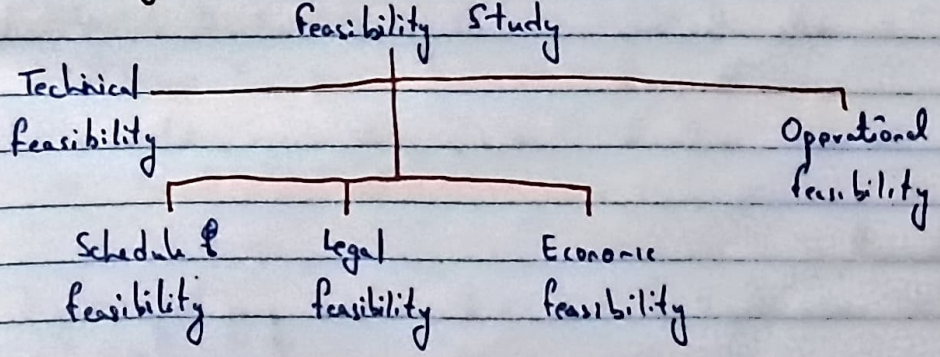
Alexander Deming

Plan – Do – Check – Act

Plan	Do	Check	Act
Identifying Problems / oppurtunities, define objectives and plan changes Planning Estimation Requirements Design	Implement on a small scale Implementation	Measure results and compare against expectations Testing Deployment	Standardise success or adjust & iterate Retrospect [Maintenance, Documentation, Changes]

1 PDCA cycle ≈ 1 iteration

Feasibility:-



Determine whether the proposed system is viable & worth pursuing

Requirements Engineering:-
Elicit, analyse, specify, validate & manage stakeholder needs so that the system built meets the intended purpose & constraints

SRS – Software Requirement Specifications

PRD – Product Requirement Document

Consists of

- ↳ req gathering & analysis (from diff povs)
- ↳ req specification
- ↳ verification & validation
- ↳ req management & change control

Design :-

Transform reqs into a robust, maintainable and implementable architecture & detailed designs that guide dev & testing.

↓

High level design (Arch) → ADD (Arch desc doc)

- ↳ decompose the system into modules / components
- ↳ represent invocation relationships among modules / components
- ↳ specify constraints

Detailed Design → DD (Design Doc)

- ↳ UML
- ↳ data structures, class diagrams, interfaces & modules are designed
- ↳ API contracts, seqs, state diags, database schema.

Implement :-

- ↳ code front & back end in some prog lang

Testing :-

Test plan ---> Implementation

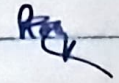
↓ !---> Design

Test !--> RE

Oracle

Dilbert!
PhD Comics!

V-Model



Maintenance :-

- Prevention → Defection prevention
- Correction → Bug fixes
- Perfection → improvements / enhancements
- Adaptive → Port software to new environments

Process Models :-

Traditional :- (plan driven)

- Waterfall model
- Prototyping
- Evolutionary
- Spiral model etal

Agile :-

- eXtreme Programming (XP)
- Scrum
- Crystal
- Feature Driven Development (FDD)

Water fall model :- → Stability !

Seq process

Req. design, implementation, testing & maintenance need to be followed in that order

Each phase needs to be completed before moving on.

Challenges :-
- ↳ Heavy weight proc
- ↳ Doc intensive
- ↳ Minimal consumer involvement after req phase
- ↳ Less flexible design
- ↳ Big bang approach to coding & integration
- ↳ One shot delivery opportunity
- ↳ Limited opp for proc improvement

Good for systems with stable requirements

Prototyping Model → Clarity !

Before starting actual dev, working prototypes are built to elicit custome. feedback.

toy model ↵
limited perf & efficiency.

- ↳ Throwaway
- ↳ Evolutionary.

Good for sys with under req, & with focus on UI/UX

Morning!

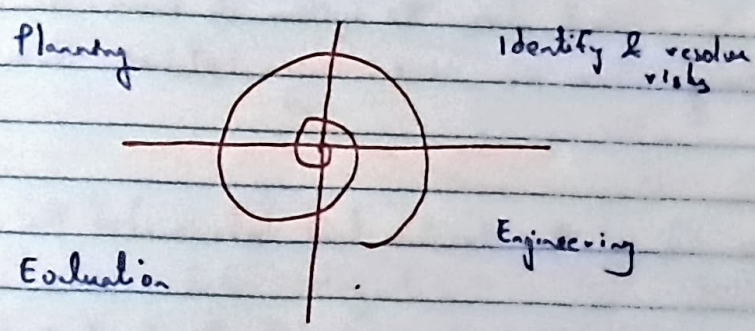
Evolutionary process model → Speed!

Sys is broken down into several modules which can be incrementally implemented & delivered.

Iterative incremental model → Model Agile

Spiral Model → Safety!

Strong emphasis on risk.
Cont. cust feedback
· Suitable for large, complex high risk systems
Expensive but highly controlled.



Each model as each loop of the spiral might be a process by itself.

Agile :-

1990 - 2000

→ Close collaboration blw dev & business experts
• 12 practices in the manifesto.
→ Maximise face-to-face communication (keep customer in the loop always)
→ Self organising teams, everyone knows what they are doing and are owners themselves

Many models under Agile

Incremental & Iterative in nature

Characteristics:-

→ Short, but many quick releases
→ Planning based on user stories
→ Each iteration deals with all life cycle activities
→ Testing ⇒ unit testing for deliverables; acceptance tests for each release
→ Flexible Design ⇒ evolution vs big upfront effort
→ Reflection after each cycle
→ Several technical & customer focused presentation opportunities

User Stories:- What does the system do from the user's pov? ~~Many~~ One or more user stories form a feature

Refactoring:- Change the structure (not behaviour) of the code

Dead Code:- Code that is very difficult to test / not possible to test at times

Open-Closed Principle:- Code is open only for extension, closed for modification

SOLID principles for OOP

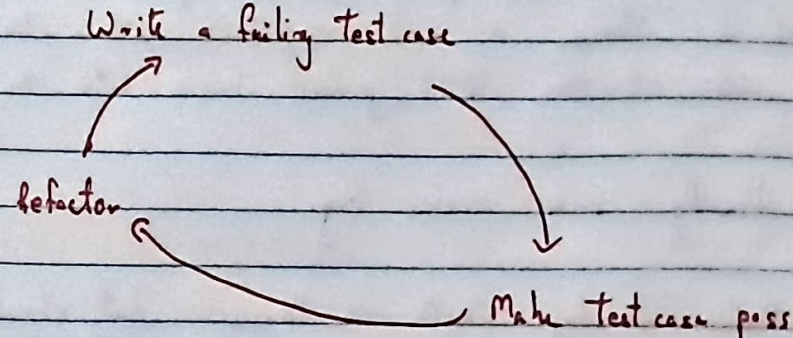
Results in Technical Debt → debt incurred cause of bad coding practices.

Code Smell ← Bad code (Ugly practices)

Long methods violate single responsibility principle
↳ Modularize

Test Driven Development:-

Write Tests First!
Test drives the development process
Write and Run testcases.
Then write code to ensure testcases pass.



Source Code Integration

Big Bang Integration is bad

Source code (download from version control)
Resolve dependencies
Compile & Run
Manual Testing
Package & Deploy.

Continuous Integration → ~~Devops~~ Development practice where devs merge changes into a shared repository frequently.

Commit → Build → Test → Feedback

Automated ↻ → Occurs continuously

Build Tools help with build process (MAVEN, ANT)

CD → Continuous Deployment (Automated)

↳ Continuous Delivery (Manual)

SCRUM Process:-

Backlog of User Stories

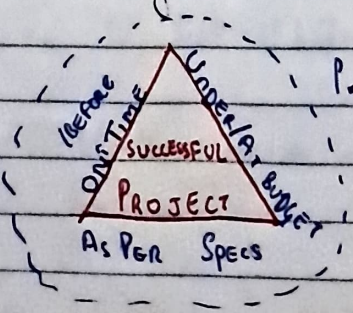
SCRUM master splits the backlog into multiple sprints.

If a story is not completed within the sprint, ~~and at the~~ it is pushed back into the backlog.

Usually 2-4 weeks long.

Daily Standup Meets → Discuss what should be done, what went well, etc to stay ~~in the~~ updated

Project Management



Faith need not be estimated.

For estimation

1) Always give a range, never a number

Experience from previous endeavours helps in making more accurate ~~[illegible]~~ estimate :-

person hour → ● work done in 1 hour by 1 person
day = 8 person hours
week = 40 person hours
year = 2000 person hours

2) Always ask what the estimate will be used for.
3) Estimation != Commitment
4) Measure, count and compute (try to atleast). Estimate only when necessary
5) Aggregate independent estimates
↳ SLLN → Planning Poker.

Methods of Estimation :-
(i) Top - down
(ii) Bottom - up
(iii) Analogy
(iv) Expert Judgement
(v) Priced to Win
(vi) Parametric / Algorithmic

(i) Function Point Analysis -
Software size measured by number & complexity of the fn it performs.
More methodical than LOC counts

② Count no of fns per category

③ Estimate complexity of fns and apply a weighting multiplier

(c) Compute an influence multiplier / value adjustment factor and apply

(d) Results in fn point total

(e) Estimating effort & time.

Types of FPA

External Inputs
User / process inputs that creates or updates data.

Data-entry screen, API endpoints etc

External Outputs
Outputs sent outside the application that contain calcs.

External Inquiries
Online queries that retrieves data with little to no processing

Internal Logical Files
Application maintained logical data groups

External Interface Files
Read only logical data used by an application but stored externally

Assign Complexity Weights

For example

1 story ≈ 1 FP

	Input	Output
Low complexity	3 FP	4
Avg	4	5
High	6	7

Refer Slides

- Scheduling and Tracking

① Partition your project
Work Breakdown Structure
↓
Checklist of the work that must be accomplished to meet the project objectives
↳ Product / Process / Hybrid WBS
Fix engine ↳ Both Product & Process
Req. Analysis, Design, Testing

Precedence Network — task
Shows the dependency of one on the other.
↓
Identify the critical path → projects on it can have no slack.
↳ Slack Time ← Max allowable delay for a non critical activity.

Requirements Engineering
↓
statement that specifies what system must do, what qualities it must possess to satisfy stakeholder needs

Main reason why projects fail

Traceability :- Very important

Root Cause Analysis for bugs

Is it possible to be absolutely certain of the verity of having correct requirements

Open systems :- Req keep changing (maybe unconstrained)

Closed systems :- (Very) Fixed requirements

We must define the scope by having a more precise problem.

Learn to say No

Scope Creep ← Scope keeps increasing

	Req must be determined	Clients produced req
New Development	A	B
Evolution of an existing system	C	D

↳ Clearly stating reduces scope creep

(i) Functional Requirements ← features / what should the system do (eg expressed as functions

(ii) Non-functional Requirements ← qualities / characteristics of the system which cannot be expressed as frs

↳ Clear specific specifications can help reassure how well the system is doing what it is supposed to do.

RE

R. Dev.
- Elicitation
- Analysis
- Negotiation
- Documentation
- Validation

R. Man
- Change Control
- Version Control
- Status Tracking
- Tracing

MVP :- Minimum Viable Requirement Product

MoSCoW method :- Must have, Should have, Could have, Won't have → Misuse cases opposite of user stories

Other methods
(i) Interviews
(ii) Observation
(iii) Workshop / Joint Application Development

~~Quality Control :-~~

Kano :- Basic & Differentiators & Delighters (Unique Selling Point)

Basic → Needs → Minimum Viable Product [MVP]

Differentiators & Delighters → Wants [frequently non-functional requirements]

Good requirements engineering reduces rework.

apscet

Date: __ __ __

Requirements Quality Control

Check if the identified requirements represent all expectations of the client

Validation :- Am I building the right product?

Verification :- Am I building the product right?

↳ Clearly understand the user req
↳ Ensure quality characteristics are met.

* Correctness
* Completeness
* Unambiguity
* Feasibility
* Necessity
* Priority
* Consistency
* Concise
* Traceability
* Standard Conformity
* Verifiability

MDD ← Model Driven Development

SRS document
One way of specification engineering
Could use propositional logic, but we end up using English

Our main requirements are functional

Non-fn requirements are tied directly or indirectly to fn reqs
↳ system level reqs
↳ Tied to multiple fnc reqs

Transitioning from Problem to Solution

Wants / Needs → What must the system do? → How should the system do it?

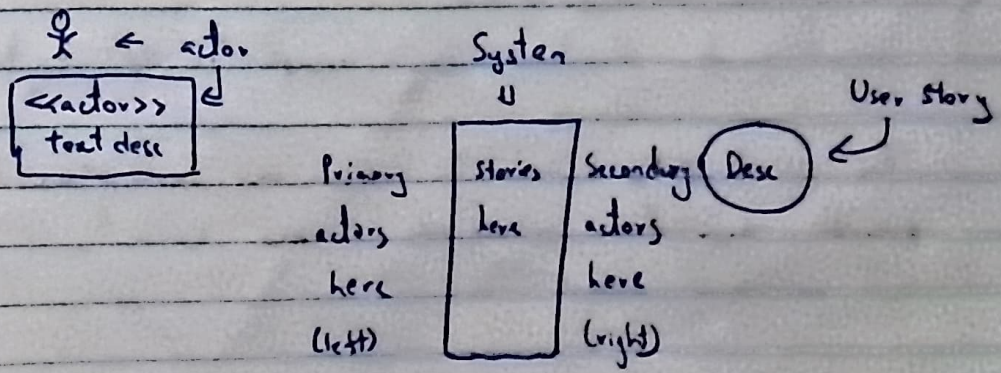
Requirements Modelling :-
Transition from Textual requirements → visual design
Helps in validating requirements.

(i) Use Case Modelling :-
↳ a seq of actions including variants that a system or any other entity can perform, interacting with the actors of the system

Focusses on what to do not how to do it.

Actor :- Initiates specific behaviours of the system. Role played by entities interacting with the system.
↳ system, subsystems, classes

System / Subject Boundary :- Boundary blw subject & actors



(i) Identify actors ← External the internal
(ii) Identify goals ← what does the user want to achieve
(iii) Define use cases ← One per goal.

There is a use case template

Post conditions ← Implementation of behaviour

Scenario:- Specific seq or path in a use case
Use case instance:- execution of scenario

Per system design.

We must now define the use cases in detail

Needs to contain certain basic information.

Use Case Name
Description
Actors
Pre-conditions
Main flow
Alt. flow
Post Conditions

Conceptual Model :-
Identify & structure key concepts in the system's domain

The N & NPs in the Use Case description list the possible candidate conceptual classes
Not software classes

primition

actual state of specific candidate class

Association; direction reading arrow

no arrow sans direction of reading the label

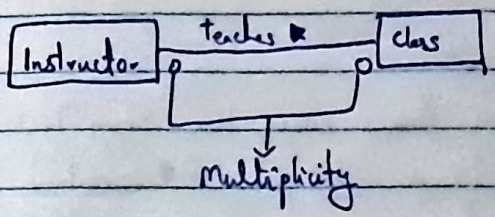
Identify attributes & relationships blw classes
Validate into for completeness & realism

Private ⇐ -

Public ∈ +

Protected ∈ #

[a..b] ← at least a, at most b occurences



U M L ← Unified Modelling Language

Testing - Guest Lecture

Test:- Activity in which a system or a component is executed under specified conditions, the results are observed & recorded & an evaluation is made of some aspect of the system

Test Case:- A set of preconditions, inputs & expected results, developed to drive the execution of the system.

Exhaustive Testing:- Test every possible input to every component of your system.

Testing based on equivalence classes:-
Class:- Group of like objects
Equivalence Classes:- Classes that are (expected to be) treated equally by the system

If we assume that every input in an equivalence class is treated the same, then testing one input in the group is sufficient

But this fails in case of inputs with limited scope (boolean, etc.)
We also need to test combinations, boundaries, 0 values [edge cases] and also perform some random tests.

Based on Coverage :-
1) Statement coverage (has every line been tested once)
2) Condition coverage (have all paths of the conditions been covered)
3) Decision coverage (more general than condition)
4) Branch coverage (Decision + implicit conditions)
5) Modified Condition Decision Coverage (Every input is toggled independently)

Form Testing :-
Testing should always gather evidence, and testing should not be done by the same team as the developer

Why Test?
- Defects
- Evidence for conformance to
  - Specs
  - Stds
  - Regs
- Safe & Unsafe scenarios to use a product
- Assess the quality
- Support for decision making

Types of Testing:-

- Unit
- Integration
- System
- Acceptance

Black box & White box

↓ ↓

You only know the requirements to be met

You know what to test exactly

All levels supported

For system level

Design Level

A B C D

↓ ↓ ↓

Fail only once in 1 bil.

All decision coverage is sufficient on a plane

Statement

∃ a format to write a test case.

Contains

- a unique id #
- objective
- priority
- traceability
- pre condition #
- inputs #
- expected outputs #

Test procedure:- Multiple sets of test cases.

Collection of test cases that need to be executed in a particular

order.

Mutation Testing:- To check for bugs in the very design of the system

Unit:- That which we choose to not divide further

Testing:- involves testing in isolation, diff test env than the final one, fast regression, stubbing, test cases only target a unit. The coverage is local. Benefits :- Changes become easy to handle, integration becomes simpler, good documentation, helps in ~~clarifying~~ clarifying specs and any improvements.

Assigning random consts to variables not being tested.

System:- Collection of units.

Local Coverage:- Inputs can be manipulated to get 100% coverage of a unit, but doesn't work when the unit is a system.

Global Coverage:- Testing of the units integration within a system

Effective

Writing ↑ Unit Tests

- ↳ Understanding Requirements
- ↳ Complex Logic
- ↳ Dependencies (Mocking / Stubbing)
- ↳ State Management.
- ↳ Test Maintainability
- ↳ Performance Overhead
- ↳ False Positives / Negatives
- ↳ Test Coverage
- ↳ Testing Edge Cases
- ↳ Maintaining Test Independence

Pack my box with five dozen liquor jugs

apsara

Date:__.__.____

Integration Testing:- ↱ Like checking correct data type rather than correct data

When more than one unit is tested to verify it's; is being transferred working ~~correctly~~ (~~correctly~~ correctly interacting with each other):

(i) Big-Bang → Integrate all at once
(ii) Top-Down → Integrate stubs first, then finalities
(iii) Bottom-Up → Integrate subsystem by subsystem Illelly
(iv) Critical first → Single Point of Failure
(v) Function at a time → All units available, do fn by fn
(vi) As-Delivered → As & when a unit is delivered
(vii) Sandwich → Mix of (ii) & (iii)

System Testing:- Final system is tested to check compliance with the fn requirements and & non-fn requirements.
Uses final real env, tests as both user and designer, checks performance - speed, latency, loads, and tests how failures are handled.
Last pt where ~~faith~~ failures are accepted

Acceptance Testing:- The customer & end user tests the system
Beta, Crowd, Compliance & Field Testing.

Validates:- Business reqs, user workflows, real-world usage, contract conditions, regulatory rules.

Decide whether or not to release
Executed in a non-developmental perspective